

Félicitations !

Vous avez terminé le cours 5 intitulé "Ajouter des capacités à l'aide d'outils".

Ce cours a transformé vos agents en les faisant passer de simples interlocuteurs intelligents en assistants capables d'agir. Vous avez implémenté le composant **tools** de l'équation de l'agent et donné à vos agents le pouvoir de faire des recherches sur le Web, d'exécuter du code, d'accéder à des bases de données et d'effectuer des actions personnalisées dans le monde réel.

Votre transformation :

- **Avant le cours 5** : les agents étant limités aux données d'entraînement des LLM, ils ne pouvaient utiliser que les connaissances existantes pour répondre.
- **Après le cours 5** : les agents peuvent passer à l'action, c'est-à-dire effectuer des recherches sur le Web en temps réel, exécuter du code, interroger des systèmes et intégrer votre logique métier.

Vous avez complété l'équation principale de l'agent : **agent = modèle + outils + orchestration**.

Ce que vous avez accompli

Vous avez commencé ce cours avec des agents sophistiqués dotés d'une configuration avancée. Vous le terminez avec des agents qui peuvent :

- **Accéder aux informations en temps réel** grâce à l'ancrage avec la recherche Google
- **Effectuer des calculs précis** avec l'exécution de code
- **Se connecter aux outils externes** via les serveurs MCP (système de fichiers, bases de données, API)
- **Exécuter votre logique métier** à l'aide d'outils de fonction personnalisés
- **Gérer les exceptions** de façon optimale et stratégique
- **Coordonner plusieurs outils** dans des workflows complexes
- **Déléguer à des spécialistes** en utilisant des modèles agent-as-tool (agent comme outil)

Compétences acquises :

- Bien comprendre les outils (ce qu'ils sont, comment les agents les utilisent)
- Utiliser des outils intégrés prêts pour la production (recherche Google, exécution de code)
- Se connecter aux serveurs MCP préexistants pour les outils de l'écosystème
- Créer des outils de fonction personnalisés qui sont adaptés à vos besoins
- Concevoir des instructions stratégiques pour une utilisation efficace des outils
- Gérer les erreurs des outils et coordonner les workflows multi-outils
- Utiliser des agents spécialisés comme outils pour un raisonnement complexe

Progression :

- Module 1 : Création de votre premier agent utilisant des outils et aperçu de l'intégration de l'état
- Module 2 : Utilisation des outils intégrés pour la recherche sur le Web et l'exécution de code + aperçu de l'état
- Module 3 : Connexion aux serveurs MCP externes pour les outils préexistants
- Module 4 : Création d'outils personnalisés avec intégration complète de l'état
- Module 5 : Outils combinés avec instructions stratégiques, gestion des exceptions et coordination de l'état

Améliorations apportées :

- Aides visuelles à l'apprentissage (diagrammes Mermaid dans les modules illustrant les processus des outils, les workflows et la gestion des exceptions)
- Intégration de l'état à tous les modules reliant le cours 4 (État) au cours 5 (Outils)
- Accessibilité revue : suppression des références futures, simplification des sections trop compliquées
- Exemples pratiques : workflows en plusieurs étapes, suivi de l'utilisation, personnalisation avec l'état

Votre progression

Module 1 : Comprendre les outils (environ 30 minutes)

Vous avez découvert en quoi consistent les outils et comment ils étendent les capacités agentiques au-delà des connaissances du LLM.

Concepts clés maîtrisés :

- ☒ Les outils permettent aux agents d'effectuer des actions allant au-delà de la simple génération de texte
- ☒ Le processus en cinq étapes des outils : raisonnement → sélection → appel → observation → finalisation
- ☒ Trois types d'outils : outil intégré, outil de fonction personnalisé, outil agent-as-tool
- ☒ Le paramètre `tools` et la manière dont les LLM utilisent les métadonnées de fonctions
- ☒ Aperçu : comment les outils peuvent lire et écrire l'état de session pour la personnalisation

Insight clé : "Les outils sont des composants de code modulaires qui exécutent des tâches prédéfinies, ce qui permet aux agents d'accéder à des informations en temps réel, d'influencer des systèmes externes et de dépasser les limites des données d'entraînement."

Module 2 : Outils intégrés (environ 25 minutes)

Vous avez appris à utiliser les outils intégrés prêts pour la production d'ADK sans écrire de code d'intégration complexe.

Concepts clés maîtrisés :

-  L'outil de recherche Google fournit des informations Web en temps réel avec ancrage
-  L'outil d'exécution de code permet d'effectuer des calculs précis et de traiter les données
-  L'ancrage permet de relier les réponses à des sources fiables
-  Exigence du règlement : obligation d'afficher les suggestions de recherche
-  Limite actuelle : un seul outil intégré par agent racine
-  Aperçu : intégration de l'état avec les outils intégrés

Insight clé : "Utilisez les outils intégrés pour les capacités courantes (recherche, exécution de code). Utilisez les outils personnalisés pour la logique spécifique à votre entreprise. Les outils intégrés sont gérés et optimisés par l'équipe ADK."

Module 3 : Outils MCP (environ 25 minutes)

Vous avez appris à connecter les agents ADK aux serveurs MCP externes pour les fonctionnalités d'outils préexistantes.

Concepts clés maîtrisés :

-  Le protocole MCP (Model Context Protocol) assure une connectivité standardisée aux serveurs d'outils
-  `McpToolset` connecte les agents ADK aux serveurs MCP
-  Deux types de connexion : `StdioConnectionParams` (locale) et `SseConnectionParams` (distante)
-  `tool_filter` contrôle quels outils sont exposés à l'agent
-  L'écosystème MCP fournit de nombreux outils prédéfinis (système de fichiers, bases de données, API)
-  Adoption par l'industrie : Anthropic, OpenAI, Google et la Linux Foundation

Insight clé : "Les outils MCP vous permettent d'exploiter l'écosystème. Utilisez des serveurs d'outils préexistants et gérés au lieu de tout concevoir vous-même."

Module 4 : Outils de fonction personnalisés (environ 35 minutes)

Vous avez appris à écrire des outils de fonction personnalisés efficaces, avec des signatures, des indications de type et des docstrings appropriées.

Concepts clés maîtrisés :

-  Signatures de fonction : noms descriptifs, indications de type, paramètres simples
-  Les docstrings guident la sélection des outils LLM (quoi, quand, arguments, valeurs de retour)

-  Renvoyer des dictionnaires structurés avec des clés d'état pour la compréhension du LLM
-  Plusieurs outils fonctionnent ensemble grâce à l'orchestration d'agents
-  ADK génère automatiquement le schéma de l'outil à partir des métadonnées de la fonction
-  Intégration complète de l'état : workflows multi-étapes, suivi de l'utilisation, personnalisation

Insight clé : "La docstring de la fonction sert de description à l'outil et est envoyée au LLM. Une docstring complète est essentielle pour que le LLM comprenne comment utiliser l'outil efficacement."

Module 5 : Combiner efficacement les outils (environ 30 minutes)

Vous avez appris à concevoir des instructions stratégiques qui guident les agents pour une utilisation efficace de plusieurs outils.

Concepts clés maîtrisés :

-  Modèles d'instructions : sélection des outils, traitement des résultats, workflows séquentiels
-  Gestion des exceptions : actions différentes selon le type d'erreur
-  Aperçu du modèle "agent-as-tool" (entièrement couvert dans de prochains cours)
-  Bonnes pratiques pour la coordination et la conception des outils
-  Les instructions stratégiques indiquent quand, comment et dans quel ordre utiliser les outils
-  Coordination d'état avec les outils : workflows multi-étapes, suivi de l'utilisation, logique conditionnelle

Insight clé : "Il est essentiel d'indiquer clairement à l'agent comment gérer les différentes valeurs de retour qu'un outil peut produire et de préciser si l'agent doit réessayer, abandonner ou demander des informations supplémentaires."

Applications concrètes

Application 1 : Service client d'e-commerce

Gérer les demandes de renseignements sur les commandes, traiter les remboursements, escalader les problèmes complexes à l'aide d'outils personnalisés avec des instructions stratégiques et une gestion complète des exceptions.

Application 2 : Recherche et analyse

Trouver des informations à jour avec la recherche Google et effectuer une analyse de données avec l'exécution de code (agents distincts en raison de la limite d'un seul outil intégré).

Application 3 : Assistance multispécialisée

Orienter les problèmes des clients vers des agents spécialisés en utilisant le modèle "agent-as-tool" doté d'une expertise spécifique au domaine.

Application 4 : Planification de voyage

Rechercher des vols et des hôtels, calculer des budgets à l'aide de plusieurs outils personnalisés avec une coordination séquentielle.

Points clés à retenir

1. **L'équation finale de l'agent** : agent = modèle + outils + orchestration. Les outils permettent aux agents d'effectuer des actions allant au-delà de la simple génération de texte.
2. **Les outils étendent les capacités des LLM**. Les LLM sont limités aux données d'entraînement. Les outils fournissent des informations en temps réel, des calculs précis, un accès aux données propriétaires et des actions externes.
3. **Le LLM s'appuie sur les métadonnées pour sélectionner les outils**. Il utilise le nom de la fonction, la docstring et le schéma des paramètres pour décider quel outil appeler.
4. **Décision entre les outils intégrés, MCP et personnalisés** : utilisez les outils intégrés pour bénéficier des capacités Google. Utilisez les outils MCP pour les intégrations d'écosystèmes. Utilisez les outils de fonction personnalisés pour appliquer une logique spécifique à l'entreprise.
5. **Schéma de retour basé sur l'état** : toujours renvoyer des dictionnaires avec la clé `status` ('success'/'error'). Il faut inclure des types d'erreurs spécifiques pour adapter la gestion.
6. **Les instructions stratégiques guident les outils**. Spécifiez quand utiliser chaque outil, comment gérer les résultats, dans quel ordre appeler les outils et quand escalader.
7. **Les différents types d'erreurs nécessitent une gestion différente**. "not_found" nécessite une clarification, "invalid_format" nécessite une explication et "permission_denied" nécessite une escalade.
8. **Agent-as-tool pour un raisonnement spécialisé** : lorsque les sous-tâches nécessitent des instructions ou une expertise différentes, utilisez un agent comme outil plutôt que comme fonction.

Checklist des bonnes pratiques

✓ Conception d'outils

- Noms de fonctions descriptifs (structure verbe-nom)
- Indications de type pour tous les paramètres
- Docstrings complètes (quoi, quand, arguments, valeurs de retour)
- Renvoyer les dictionnaires avec la clé d'état
- Messages d'erreur spécifiques et faciles à comprendre
- Un seul objectif précis par outil
- Types de paramètres sérialisables en JSON

✓ Outils intégrés

- Utilisez la recherche Google pour obtenir des informations en temps réel.
- Utilisez l'exécution de code pour des calculs précis.
- Affichez les suggestions de recherche (exigence du règlement).
- N'oubliez pas la limite d'un seul outil intégré par agent.
- Utilisez les modèles Gemini 2.0 ou versions ultérieures.

✓ Outils MCP

- Vérifiez le [registre de serveurs MCP](#) avant d'écrire des outils personnalisés.
- Utilisez `tool_filter` pour limiter les outils exposés.
- Utilisez `StudioConnectionParams` pour le développement.
- Utilisez `SseConnectionParams` pour la production.
- Assurez-vous que Node.js est installé pour les serveurs basés sur npx.

✓ Conception des instructions

- Faites référence aux outils par leur nom.
- Précisez quand chaque outil doit être utilisé.
- Définissez les workflows séquentiels étape par étape.
- Gérez tous les types d'erreurs avec des actions spécifiques.
- Incluez les procédures d'escalade.
- Organisez les informations en sections claires.

✓ Gestion des exceptions

- Renvoyez les types d'erreurs spécifiques.
- Fournissez des messages d'erreur lisibles par l'humain.
- Spécifiez les stratégies de nouvelle tentative et d'abandon.
- Définissez les critères d'escalade.
- Testez tous les chemins d'erreur.
- Documentez les erreurs attendues dans les docstrings.

Référence des modèles de code

Modèle 1 : Outil personnalisé de base

```

def tool_name(param: type) -> dict:
    """Résumé en une ligne. Utiliser lorsque [scénario].

    Args:
        param (type): Description

    Renvoie :
        dict: {'status': 'success'/'error', ...}
    """
    if validation_fails:
        return {"status": "error", "error_message": "explanation"}
    return {"status": "success", "data": value}

```

Modèle 2 : Utilisation des outils intégrés

```

from google.adk.tools import google_search
from google.adk.code_executors import BuiltInCodeExecutor

# Recherche Google
agent = LlmAgent(
    model='gemini-2.5-flash',
    tools=[google_search]
)

# Exécution de code
agent = LlmAgent(
    model='gemini-2.5-flash',
    code_executor=BuiltInCodeExecutor()
)

```

Modèle 3 : Utilisation des outils MCP

```

from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

agent = LlmAgent(
    model='gemini-2.5-flash',
    name='mcp_agent',
    tools=[
        McpToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command='npx',
                    args=['-y', '@modelcontextprotocol/server-filesystem', '/path'],
                ),
            ),
            tool_filter=['list_directory', 'read_file'], # Sécurité : limiter les
outils
        )
    ],
)

```

Modèle 4 : Outil multifonction avec des instructions stratégiques

```
agent = LlmAgent(
    instruction="""
    ## Sélection des outils
    - Utiliser tool_1 lorsque [scénario]
    - Utiliser tool_2 lorsque [scénario]

    ## Workflows
    Pour [tâche] :
    1. Commence par utiliser tool_1.
    2. Ensuite, utilise tool_2.

    ## Gestion des exceptions
    - "not_found" : demande à l'utilisateur de vérifier.
    - "invalid_format" : explique le format correct.
    """,
    tools=[tool_1, tool_2]
)
```

Modèle 5 : L'agent comme outil

```
from google.adk.tools.agent_tool import AgentTool

specialist = LlmAgent(
    name='specialist',
    instruction="Expertise spécifique au domaine..."
)

main_agent = LlmAgent(
    instruction="Fais appel à un spécialiste lorsque [scénario]",
    tools=[AgentTool(agent=specialist)]
)
```

Erreurs courantes à éviter

✗ Conception d'outils

- Noms de fonctions vagues (`process()` au lieu de `process_refund()`)
- Indications de type manquantes (le LLM nécessite des informations de type)
- Docstrings insuffisantes (le LLM s'appuie sur elles pour sélectionner les outils)
- Renvoyer des chaînes plutôt que des dictionnaires (les données structurées sont plus claires)
- Outils polyvalents (axés sur un seul objectif)

✗ Instructions

- Trop vagues (ne renseignent pas sur l'utilisation des outils)
- Aucune indication concernant les erreurs (le LLM ne sait pas comment gérer les

- défaillances)
- Séquences manquantes (pas d'ordre pour les workflows multi-outils)
- Aucune procédure d'escalade (l'agent est bloqué)

✗ Outils intégrés

- Combiner des outils intégrés et personnalisés dans le même agent (impossible)
- Version du modèle incorrecte (nécessite Gemini 2.0 ou version ultérieure)
- Ignorer le règlement concernant les suggestions de recherche (affichage obligatoire)

✗ Outils MCP

- Ne pas vérifier le registre de serveurs MCP avant d'écrire des outils personnalisés
- Exposer tous les outils sans `tool_filter` (risque de sécurité)
- Prérequis Node.js manquant pour les serveurs npx

✗ Gestion des exceptions

- Erreurs génériques renvoyées (impossible de différencier les défaillances)
- Aucune aide apportée aux utilisateurs en cas d'erreur (pour résoudre les problèmes)
- Boucles de répétition de tentatives sans modifications (même entrée = même échec)
- Absence de documentation des types d'erreurs dans les docstrings

Mémento

Tableau comparatif des types d'outils

Type	Configuration	Application idéale	Maintenance
Intégré	Importer depuis ADK	Capacités courantes	Équipe ADK
MCP	Se connecter au serveur MCP	Outils préexistants de l'écosystème	Responsables de la maintenance des serveurs
Fonction personnalisée	Écrire une fonction Python	Logique métier	Vous
Agent-as-tool	Créer LlmAgent	Raisonnement complexe	Vous

Quand utiliser l'un ou l'autre ?

Scénario	Méthode	Pourquoi
Recherche sur le Web	Intégrée : <code>google_search</code>	Prête pour la production, gérée

Scénario	Méthode	Pourquoi
Calculs	Intégrée : <code>BuiltInCodeExecutor</code>	Exécution de code sécurisée
Capacité préexistante	Outils MCP	Maintenance par la communauté
Votre logique métier	Outil de fonction personnalisé	Vos besoins spécifiques
Raisonnement complexe	Agent-as-tool	Adaptation nécessaire

Guide rapide de gestion des exceptions

Type d'erreur	Action	Exemple
<code>not_found</code>	Demander à l'utilisateur de vérifier	"Pourriez-vous vérifier l'ID de commande ?"
<code>invalid_format</code>	Expliquer avec un exemple	"Les ID de commande commencent par 'ORD' (par exemple, ORD123)."
<code>permission_denied</code>	Escalader immédiatement	"Je vais vous mettre en relation avec un superviseur."
<code>temporary_failure</code>	Réessayer une fois, puis escalader le problème	"Je vais réessayer... Le problème persiste."

Importations essentielles

```
from google.adk.agents import LlmAgent
from google.adk.tools import google_search
from google.adk.code_executors import BuiltInCodeExecutor
from google.adk.tools.agent_tool import AgentTool
from google.adk.tools.mcp_tool import McpToolset
```

Communauté et support

Principales références

- [Présentation des outils personnalisés](#) - Guide complet de création d'outils personnalisés dans ADK
- [Outils intégrés](#) - Recherche Google, exécution de code et autres outils prêts pour la production
- [Intégration des outils MCP](#) - Intégration des outils Model Context Protocol
- [Outils de fonction](#) - Documentation détaillée sur les signatures de fonction, les docstrings et les modèles

- [Ancrage avec la recherche Google](#) - Utilisation de l'outil de recherche Google avec ancrage et citations

Ressources MCP

- [Registre des serveurs MCP](#) - Catalogue officiel des serveurs MCP
- [Documentation MCP officielle](#) - Guides et spécifications du protocole
- [GitHub MCP](#) - Implémentations de référence

Ressources

De la communauté

- [Tools Make an Agent: From Zero to Assistant with ADK](#) - Blog officiel de Google Cloud offrant une vue d'ensemble des outils de fonction, des outils intégrés et des modèles d'orchestration d'outils
- [Google ADK Masterclass Part 2: Adding Tools to Your Agents](#) - Tutoriel détaillé sur l'ajout d'outils aux agents avec des exemples concrets et des bonnes pratiques
- [Building AI Agents with Google's ADK: Part 2 — Function Calling](#) - Présentation détaillée des outils de fonction personnalisés, y compris les signatures, les docstrings et les schémas de retour
- [A Guide to ADK Tools: Leveraging Function Tools and Built-in Tools](#) - Guide pratique avec des exemples concrets pour les outils intégrés et personnalisés
- [Use Google ADK and MCP with an external server](#) - Guide officiel pour intégrer les outils MCP (Model Context Protocol) en vue d'une intégration avancée

Des questions ? Posez-les sur le [forum de la communauté](#).

Étape suivante

- **Cours 6 : Ajouter des garde-fous avec des rappels** - Implémenter la surveillance, la sécurité et le contrôle

Conclusions

Vous avez commencé ce cours avec des agents capables de penser et de répondre intelligemment. Vous le terminez avec des agents qui peuvent **agir** dans la vraie vie.

Votre transformation est significative :

Avant : des agents limités par les données d'entraînement des LLM. Après : des agents capables d'effectuer des recherches sur le Web en temps réel, d'exécuter du code pour des

calculs précis, d'accéder à vos bases de données, d'appeler vos API et d'appliquer votre logique métier. C'est toute la différence entre un chatbot et un assistant.

Les compétences que vous avez acquises sont fondamentales :

Conception stratégique des instructions, modèles de gestion des exceptions, coordination des outils et modèle agent-as-tool : il ne s'agit pas seulement de techniques enseignées pour les besoins de ce cours. Ce sont des principes fondamentaux que vous appliquerez tout au long du parcours de formation d'ADK et dans chaque agent que vous créerez.

L'équation principale est complète :

Agent = modèle + outils + orchestration

Vous comprenez le modèle (cours 2-3). Vous avez implémenté des outils (cours 5). Vous maîtriserez l'orchestration dans les cours suivants.

Vous avez des questions ou des commentaires ? Faites-nous en part sur le [forum de la communauté ADK](#).

Prêt à relever un nouveau défi ? Dans les prochains cours, vous apprendrez à surveiller, contrôler, coordonner et faire évoluer les agents utilisant des outils que vous êtes désormais à même de créer.